The Next Generation of ReShare

Architecture, adaptable workflows, interoperability, and migration

Jakub Skoczen · Head of Engineering, Index Data · WOLFcon 2026

From CrossLink broker to CrossLink platform

WOLFcon 2025	WOLFcon 2026
Standards-based ILL broker	Complete ILL workflow platform
Connect external peers	Native borrowing and lending
Route and observe transactions	Model and operate the lifecycle

New: Patron Requests API · NCIP · explicit state model · live UI events · scheduling

From a proven service to the next generation

What ReShare proved

- Community-owned resource sharing works
- ILS-neutral workflows can operate in production
- Practitioners can shape the service
- Open standards connect diverse systems

What needed to change

- Substantial application and persistence framework overhead
- Tenant topology multiplied operational work
- Kafka added infrastructure and failure windows
- Workflow behavior was spread across the application
- Workflow changes required tracing, releases, and broad testing

We are replacing an implementation, not the product.

Requirements before technology

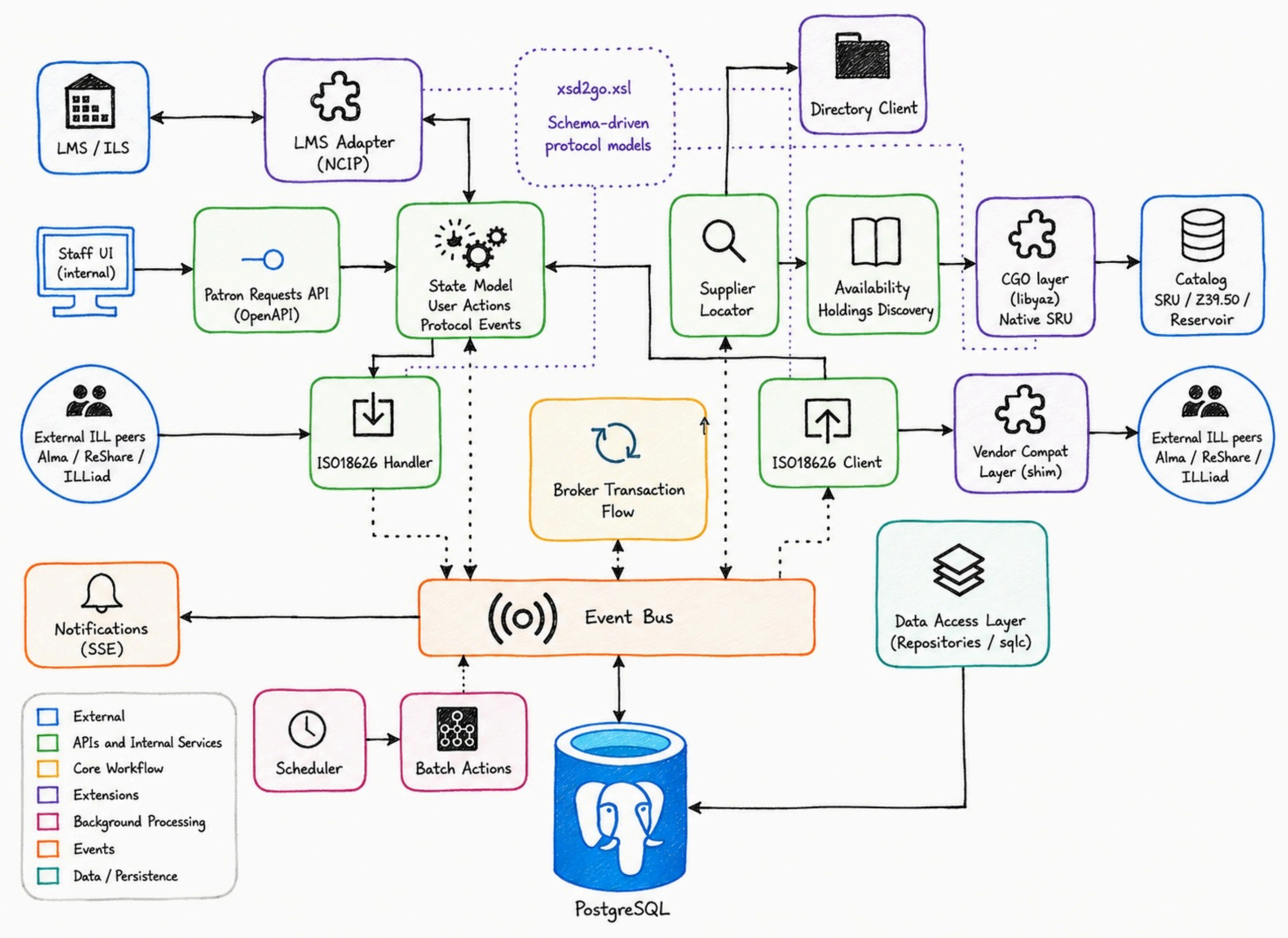
Preserve

- ILS/LSP neutrality
- First-class ISO 18626 and NCIP
- Borrowing and lending workflows
- Consortial routing and supplier selection
- Multi-tenant operation

Improve

- Smaller, more reliable operational footprint
- Explicit and adaptable workflows
- Observable and auditable requests
- One shared deployment for virtual tenants
- Support for native direct borrowing across the consortium
- A richer, OpenAPI-based Directory for networks, tiers, policies, and integrations (NCIP, Z39.50)

A platform built around the transaction



Open protocols at the edges · explicit workflow at the center · durable work underneath

A small core supports a complete platform

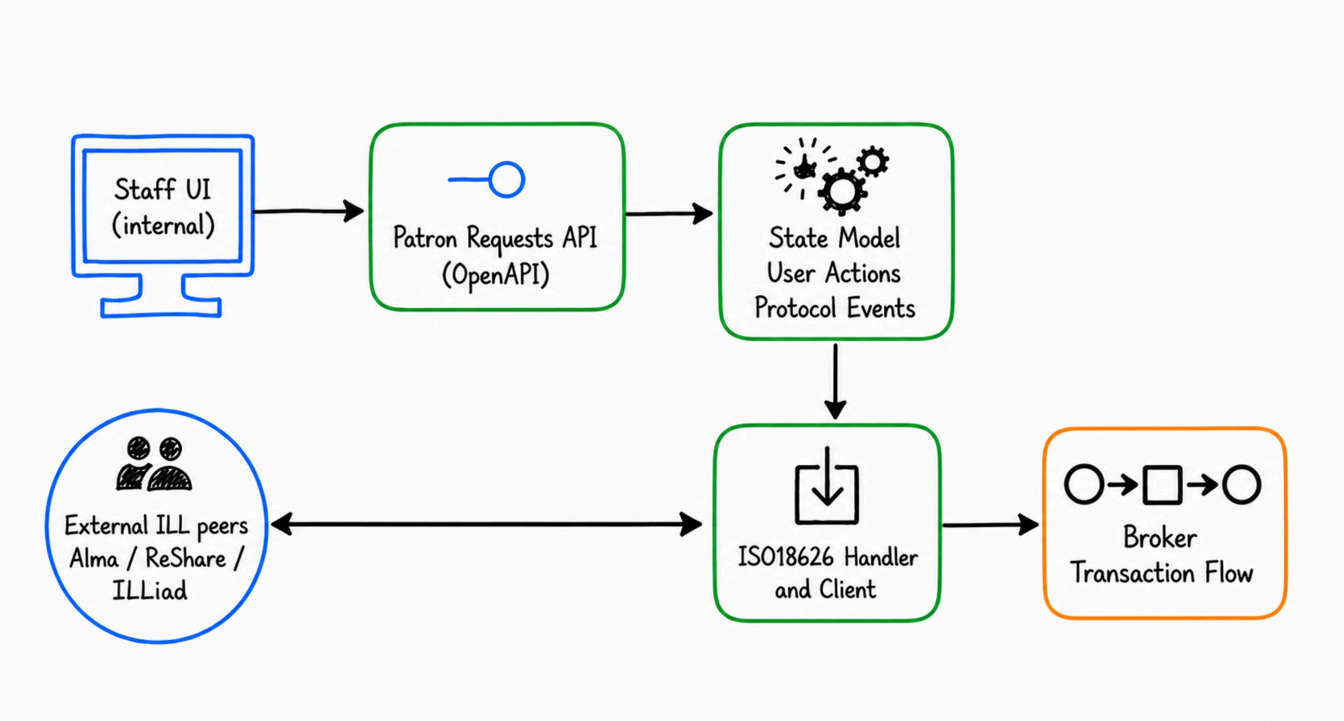
Lightweight core runtime

- One Go service around durable state
- PostgreSQL as the only infrastructure dependency
- No Kafka cluster for core messaging
- Custom event bus implementation
- Stateless Kubernetes/Helm deployment and health endpoints
- Packaged database migrations and rolling updates

Main features built on the shared core

- Workflow execution
- Scheduler and batch actions
- Request aging and retries
- Notifications and email templates
- Pull slips and document delivery
- Live UI updates through SSE

One request, two complementary views



Patron Request

- The lifecycle practitioners operate
- Borrowing or lending perspective
- Current state and available actions
- Items, notifications, and attention flags
- Search, filtering, and batch work

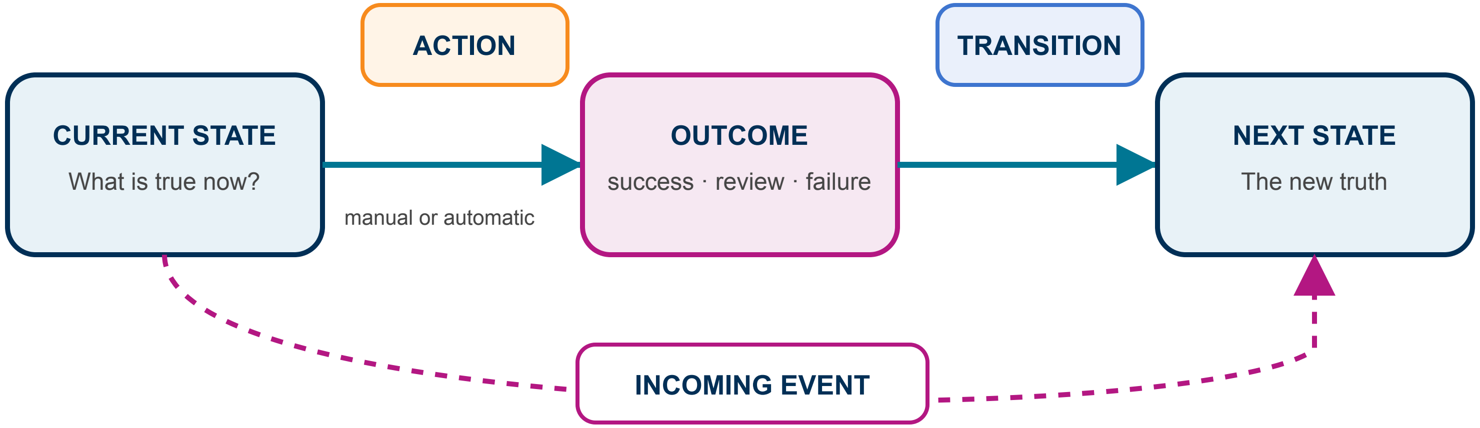
ILL Transaction

- Supplier and rota decisions
- ISO 18626 message exchange
- Adapter and protocol activity
- Complete durable event history

A hyperlinked API connects the request, transaction, protocol activity, and event history—from intent to outcome.

Workflow is now a YAML model

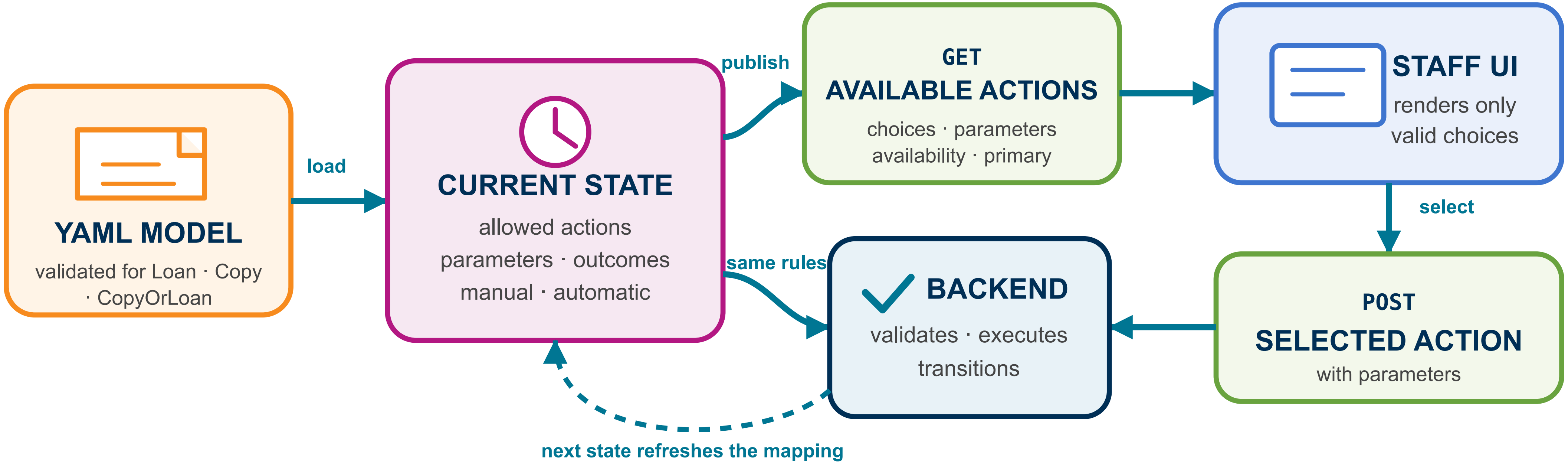
A small grammar for a complete workflow



External messages use the same permitted transitions

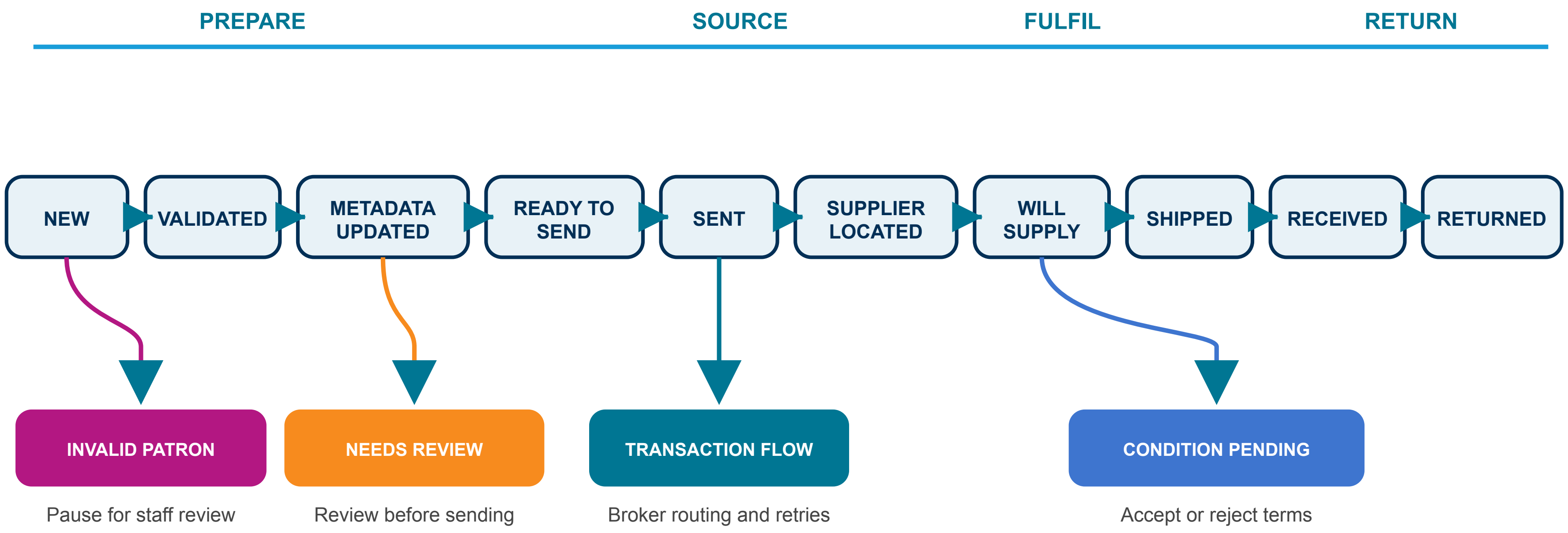
```yaml
- name: NEW
  side: REQUESTER
  initial: true
  primaryAction: validate-patron
  actions:
    - name: validate-patron
      trigger: auto
      transitions: {success: VALIDATED, review: INVALID_PATRON}
```

One workflow model drives the API and UI



Accepted actions become durable database events: recoverable after failure, observable through the API, and safely distributed across broker instances.

A borrowing request, step by step



The happy path is simple. The model makes exceptions explicit.

The happy path and every exception are explicit, inspectable, and testable.

Customizable with guardrails

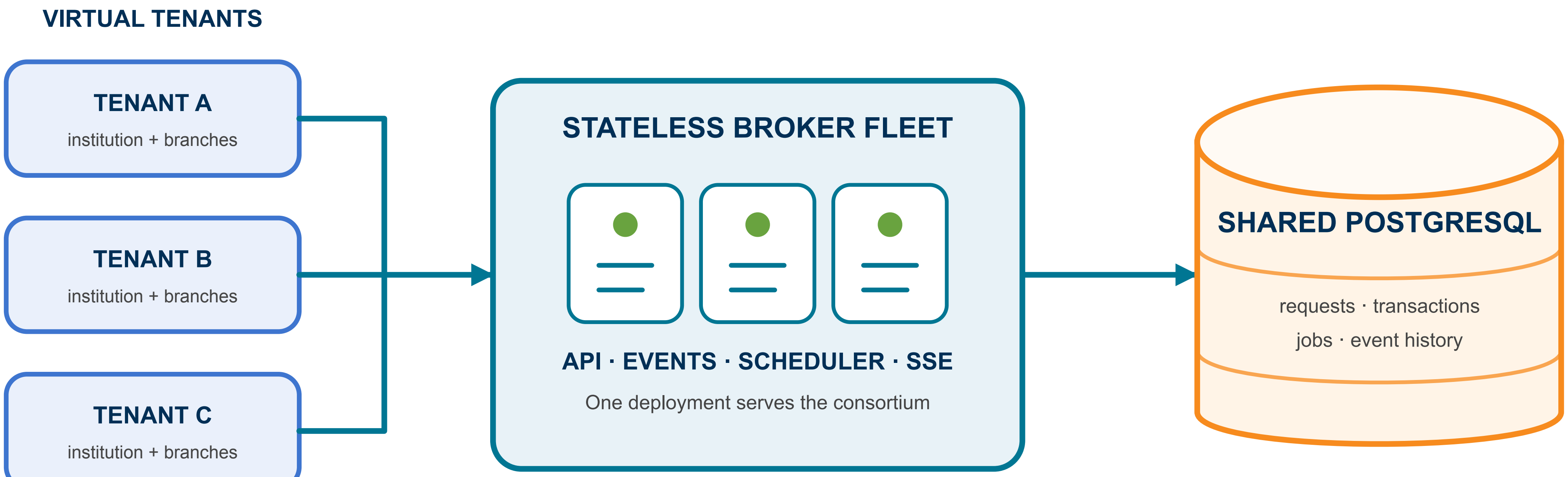
Shape the workflow

- Choose supported actions per state
- Make actions manual or automatic
- Map outcomes and incoming messages
- Apply behavior by service type
- Mark primary, editable, terminal, and attention states

Enforce the contract

- Publish supported capabilities
- Reject unknown actions and events
- Prevent requester/supplier crossovers
- Validate transitions and closing paths
- Check service-type consistency

One deployment, many virtual tenants



Identity, ownership, and policy —not infrastructure

Scale replicas for total demand

One database and schema

The new architecture enables native DCB support in ReShare
Union catalog → live availability → internal fulfillment → local ILS

A shared deployment is an option, not a constraint
Independent local brokers can still form a peer-to-peer ILL network

Lightweight is measurable

Measure	CrossLink	mod-rs	Difference
Container image	35.6 MB	246.0 MB	**6.9× smaller**
Slowest cold start—3 runs	0.667 s	37.006 s	**55× faster**
App working set (RSS)	19.7 MiB	1,509.2 MiB	**77× lower**
Core runtime pieces	2	5	**3 fewer**
Test-to-production LOC ratio	1.71:1	0.16:1	**10.9× higher**

Native ARM and a 2 GiB app limit. Legacy app tier includes Kafka + ZooKeeper; PostgreSQL and Okapi excluded.

A smaller operational footprint means simpler deployment, recovery, scaling, and upgrades.

Adoption with a migration plan

OPTION A · TRANSFER ACTIVE REQUESTS — **BOUNDED MAINTENANCE WINDOW**

1. **FREEZE LEGACY** — pause staff writes and new submissions
2. **FINAL EXPORT AND IMPORT** — capture the final delta
3. **RECONCILE** — counts · states · IDs; resolve exceptions
4. **SWITCH INTEGRATIONS** — VuFind · ISO 18626 · NCIP
5. **SMOKE TEST AND OPEN** — enable live traffic and monitor

OPTION B · FINISH ACTIVE REQUESTS IN LEGACY — **NO LIVE-REQUEST MIGRATION**

LEGACY RESHARE
LIVE BEFORE CUTOVER
holds the current requests

→ **CONTROLLED CUTOVER**
route new requests to Crosslink
retain what legacy needs to finish

→ **LEGACY RESHARE**
FINISH EXISTING REQUESTS, THEN RETIRE

→ **CROSSLINK**
ALL NEW REQUESTS START HERE

Both systems run in parallel—but each request stays in one system.

Rehearse with real data, prove readiness, and choose the cutover path that fits each consortium.

A platform that evolves with the service

- Open at the edges
- Explicit at the center
- Small enough to operate
- Flexible enough to evolve
- A migration path that protects continuity

The next generation of ReShare makes the community’s ILL workflow a first-class part of the platform.